

ARQUITECTURA DE AGENTES IA · UTDT 2026

Módulo 1

Introducción a los

Agentes IA

Agency Spectrum · Three Pillars · Foundation

Models · ADK2

UTDT 2026

MÓDULO 1 DE 13

¿Qué es un agente IA?

*Un agente IA es un sistema autónomo que combina un **modelo de fundación** con herramientas, memoria y orquestación para manejar entradas no estructuradas y tareas de múltiples pasos.*

La diferencia clave frente a sistemas tradicionales:

- **Código rígido** → entradas deterministas, lógica estática
- **Workflows** → ramas finitas, checkpoints explícitos
- **RAG / chatbot** → responde preguntas, no actúa
- **Agente** → planifica, decide, actúa, aprende

REGLA PRÁCTICA:

Si el problema tiene entradas variables, razonamiento multistep o tareas paralelas → agente. Caso

El sistema agentic completo

Un agente no es solo un modelo. Es toda la **infraestructura** que lo rodea:

MODELO DE FUNDACIÓN

- GPT-5, Claude, Gemini, Llama
- Preentrenado, usable sin fine-tuning
- Motor de razonamiento central

HERRAMIENTAS

- APIs externas, búsqueda web
- MCP (Model Context Protocol)
- Ejecutores de código

MEMORIA & ORQUESTACIÓN

- Memoria a corto y largo plazo
- Planificación de pasos
- Coordinación multi-agente

La agencia no es binaria

SCRIPT DETERMINISTA

- Sin decisiones reales
- Flujo fijo, sin variabilidad
- Ej: cron job, batch SQL

WORKFLOW IA

- Ramas condicionales
- Pasos predefinidos con IA
- Ej: clasificación + ruta

AGENTE AUTÓNOMO

- Toma decisiones reales
- Planifica, actúa, aprende
- Ej: agente de investigación

La pregunta clave: ¿El sistema demuestra toma de decisión real o simplemente sigue un script estático? La respuesta determina dónde cae en el agency spectrum.

Calibrar la autonomía al riesgo

El nivel de agencia debe **calibrarse** según dos dimensiones:

VARIABILIDAD DE LA TAREA

- **Alta variabilidad** → más agencia
- **Baja variabilidad** → más determinismo
- Medir: ¿cuántos casos edge hay?

TOLERANCIA AL RIESGO

- **Alto riesgo** → menos autonomía
- **Bajo riesgo** → más autonomía posible
- Ejemplo: finanzas vs. redacción

En ADK2 (Google Agent Development Kit 2) este principio se implementa a nivel de herramienta: cada **tool** puede marcarse con `require_confirmation` para exigir aprobación humana antes de ejecutarse (human-in-the-loop granular).

¿Cuándo usar un agente? Los 4 factores

FACTOR	CÓDIGO/WORKFLOW	AGENTE
Variabilidad de entrada	Baja, predecible	Alta, no estructurada
Complejidad de razonamiento	Lógica simple o fija	Multistep, ambigüedad
Performance / Compliance	Latencia ultra-baja, auditable	Flexible, tolerante a latencia
Carga de mantenimiento	Bajo si el problema es estable	Menor cuando el problema cambia

PRINCIPIO:

Usar siempre la arquitectura más simple que resuelva el problema. La complejidad de un agente tiene un costo real.

El árbol de decisión

¿Entrada determinista + latencia crítica + lógica auditable?

↳ CÓDIGO TRADICIONAL

¿Ramas finitas + checkpoints explícitos + pasos predecibles?

↳ WORKFLOW

¿Responder preguntas sobre un corpus sin acción autónoma?

↳ RAG / CHATBOT

¿Entradas no estructuradas + planificación multistep
+ tareas paralelas + aprendizaje continuo?

↳ AGENTE IA ✓

La distinción entre RAG y agente es frecuentemente malentendida: **RAG recupera y genera, pero no actúa ni planifica.**

Modelos de fundación: el motor del agente

Un *foundation model* es un modelo preentrenado a gran escala, utilizable directamente sin entrenamiento específico por tarea.

Características clave:

- **Transferibilidad** — un solo modelo para múltiples tareas
- **Emergencia** — capacidades no explícitamente entrenadas
- **Adaptabilidad** — prompt engineering, RAG, fine-tuning opcional
- **Multimodalidad** — texto, código, imágenes, audio (modelos modernos)

Modelos líderes en 2026: **GPT-5**, **Claude 4**, **Gemini**, **Llama** (open source)

En Google Cloud / ADK2: Gemini 2.5 Pro es el modelo recomendado como backbone por su integración nativa con Vertex AI y las herramientas de GCP.

Selección de modelo: benchmarks 2025

MODELO	HELM CORE SCORE	PERFIL
GPT-5 mini	0.819	Eficiencia + calidad
o4-mini	0.812	Razonamiento rápido
o3	0.811	Razonamiento profundo
Gemini 2.5	s/d en este corte	Integración GCP nativa
Claude 4	s/d en este corte	Seguimiento de instrucciones

s/d = sin dato en el corte de HELM Core usado (agosto 2025). No se completa con un número no verificado: los scores no son comparables entre cortes, entre variantes de configuración del mismo modelo, ni contra otro benchmark. Un leaderboard ordena candidatos; no decide. La decisión se toma midiendo sobre la tarea propia.

Recomendación de diseño:

- Comenzar con el modelo general líder (requiere mínima customización)
- Para tareas bien definidas, baja latencia o costo: modelos pequeños suelen ser suficientes
- **Tendencia 2026:** routing automático de modelos — queries simples → modelo pequeño; complejas → modelo grande

Diseño multimodelo desde el día uno

POR QUÉ ES CRÍTICO

- Los modelos mejoran cada 6-12 meses
- Precios fluctúan drásticamente
- Diferentes modelos tienen diferentes fortalezas
- Evitar vendor lock-in

CÓMO IMPLEMENTARLO

- Abstracción de la capa de modelo
- Estándares abiertos (OpenAPI)
- Routing automático por tarea (lógica de aplicación)
- ADK2: `model` es un string plano → swappable

```
# ADK2: el modelo es un string plano → swappable sin tocar lógica
from google.adk.agents import Agent

agent = Agent(
```

7 + 2 tipos de agentes IA

TIPO	EJEMPLOS	CAPACIDAD PRINCIPAL
Business-task	UiPath, Power Automate, Zapier	Automatización de workflows deterministas
Conversacional	Chatbots de soporte	Gestión de diálogo, reconocimiento de intención
Investigación	Perplexity, Elicit	Recopilación + síntesis de información
Analytics	Power BI Copilot, Glean	Datos estructurados → insights
Developer	Cursor, Windsurf, GitHub Copilot	Generación y refactoring de código
Domain-specific	Harvey (legal), Hippocratic (médico)	Asistencia experta en dominio
Browser-using	—	Navegar e interactuar con sitios web
Voice	—	Automatización de voz end-to-end
Video	—	Respuestas en video con avatares

Agentes de tarea de negocio

CARACTERÍSTICAS

- Automatizan procesos repetitivos
- Integran sistemas empresariales
- Combinan RPA con LLM
- Mayor adopción empresarial actual

EJEMPLOS

- UiPath (RPA + IA)
- Power Automate (Microsoft)
- Zapier (integración no-code)
- ADK2 en GCP + Apigee

*En el stack de Google Cloud, los agentes de tarea se construyen sobre **ADK2 + Vertex AI Agent Builder**, conectando con sistemas legacy via **Apigee** como gateway de APIs.*

Agentes de dominio específico

Los más sofisticados porque requieren **conocimiento experto verificable**:

LEGAL

- Harvey AI — revisión de contratos
- Requiere precisión absoluta
- Responsabilidad legal del output

MÉDICO

- Hippocratic AI — triage
- FDA/regulatorio crítico
- Human-in-the-loop obligatorio

SEGURIDAD

- SOC analyst agent
- Detección de amenazas 24/7
- Chronicle + Vertex AI (GCP)

MCP y A2A: los protocolos del ecosistema

MCP — MODEL CONTEXT PROTOCOL

- Permite a agentes usar **herramientas remotas**
- Estándar abierto (Anthropic, 2024)
- Adoptado por ADK2, LangGraph, OpenAI
- Profundizado en Módulo 4

A2A — AGENT-TO-AGENT PROTOCOL

- Permite **colaboración entre agentes**
- Estándar Google (2025)
- Soporte nativo en ADK2
- Profundizado en Módulo 8

Agent A —[MCP]→ Tool Server (búsqueda, base de datos, API)

Agent A —[A2A]→ Agent B (especializado, delegación de subtarea)

Sync vs. Async: la naturaleza de los agentes

	SÍNCRONO	ASÍNCRONO
Patrón	Request → Response	Publish → Subscribe / Event
Uso	APIs simples, chatbots básicos	Agentes, pipelines complejos
Bloqueo	Bloquea hasta respuesta	No bloquea, continúa
Paralelismo	Limitado	Nativo
Ejemplos	REST API simple	ADK2 async agents, LangGraph

Los agentes operan fundamentalmente de forma *asíncrona*: ejecutan tareas en paralelo y priorizan dinámicamente. Asíncrono *no* significa que no esperen — el lab de este módulo hace `await runner.run_async(...)` y espera la respuesta del modelo. Significa que esa espera *no bloquea el hilo*: el runtime sigue atendiendo otras invocaciones mientras tanto.

Los 5 principios del sistema agentico

1. ESCALABILIDAD

Arquitectura distribuida, autoscaling. En GCP: Cloud Run + Vertex AI.

2. MODULARIDAD

Componentes intercambiables via interfaces claras. ADK2: herramientas como módulos independientes.

3. APRENDIZAJE CONTINUO

Feedback loops integrados al agent loop. No set-and-forget.

4. RESILIENCIA

Retry logic, fallback, error handling. Los agentes fallan: el diseño debe anticiparlo.

5. FUTURE-PROOFING

Estándares abiertos (OpenAPI, MCP, A2A). Evitar vendor lock-in desde el diseño.

Principio 1: Escalabilidad

```
# ADK2: el agente se define una sola vez, sin código específico de infra
from google.adk.agents import Agent

agent = Agent(
    name="soporte_agent",
    model="gemini-2.5-pro",
    tools=[customer_db_tool, ticket_tool]
)
```

```
# El despliegue a Cloud Run se hace vía la CLI de ADK, no vía código Python.
# Cloud Run escala de 0 a N instancias automáticamente (lo maneja Cloud Run,
# no el SDK de agentes):
adk deploy cloud_run \
    --project=mi-proyecto --region=us-central1 \
    path/to/agent_folder -- --min-instances=0 --max-instances=100
```

En GCP: la combinación **Vertex AI + Cloud Run** permite escalar agentes desde 0 hasta miles de instancias sin configuración manual de infraestructura.

Principio 2: Modularidad

```
# Herramientas como módulos independientes en ADK2
from google.adk.tools import FunctionTool

# Cada herramienta es un módulo swappable
search_tool = FunctionTool(func=buscar_en_web)
db_tool = FunctionTool(func=consultar_base_datos)
email_tool = FunctionTool(func=enviar_email)

# El agente compone herramientas independientemente del modelo
agent = Agent(
    model="gemini-2.5-pro", # Swappable
    tools=[search_tool, db_tool, email_tool] # Swappable
)
```

La modularidad significa que cambiar el modelo no rompe las herramientas, y agregar una herramienta no requiere modificar el agente central.

Principio 4: Resiliencia

```
# Retry y timeout en ADK2 (campos reales de Agent)
from google.adk.agents import Agent
from google.adk.workflow import RetryConfig

agent = Agent(
    name="resiliente_agent",
    model="gemini-2.5-pro",
    tools=[primary_tool],
    timeout=30.0,           # segundos antes de abortar la invocación
    retry_config=RetryConfig(
        max_attempts=3,
        backoff_factor=2.0,
        exceptions=[TimeoutError],
    ),
)
```

Los agentes *fallan*. La pregunta no es si fallarán, sino cuándo. El diseño resiliente anticipa el fallo y tiene respuesta preparada.

El *fallback a un modelo de respaldo* no es un kwarg nativo del `Agent`: se implementa como lógica de aplicación (capturar la excepción una vez agotado `retry_config` y reintentar con un segundo `Agent` configurado con otro `model`, p. ej. `gemini-2.5-flash`).

Comparativa de frameworks agenticos

FRAMEWORK	MEJOR PARA	TRADE-OFF
ADK2 (Google)	GCP, producción enterprise, multiagente	Acoplamiento a Google Cloud
LangGraph	Producción single/multiagente ligero, flujo explícito	Lógica custom para memoria/planificación
AutoGen	Investigación + producción multiagente con diálogo	Pesado para casos simples
CrewAI	Prototipos rápidos, agentes human-centric	Customización limitada
OpenAI Agents SDK	Equipos en stack OpenAI, tool-using rápido	Acoplado a OpenAI

ADK2: Google Agent Development Kit 2

El framework de referencia del curso para producción:

CAPACIDADES NATIVAS

- Soporte MCP y A2A built-in
- Integración Vertex AI / Gemini
- Memory & Session management
- Retry, timeout y callbacks configurables por agente

STACK GCP

- Cloud Run (despliegue)
- Vertex AI Agent Builder (UI)
- Apigee (API gateway)
- Chronicle (seguridad)

```
pip install google-adk
adk create my-agent # Scaffold completo listo para Cloud Run
```

LangGraph: flujo explícito y control total

```
from langgraph.graph import StateGraph, END

# Definir estado y nodos del agente
builder = StateGraph(AgentState)
builder.add_node("razonar", razonar_fn)
builder.add_node("actuar", actuar_fn)
builder.add_node("observar", observar_fn)

# Flujo explícito – el desarrollador controla cada transición
builder.add_edge("razonar", "actuar")
builder.add_edge("actuar", "observar")
builder.add_conditional_edges(
    "observar",
    lambda s: "razonar" if s["continuar"] else END
)

graph = builder.compile()
```

LangGraph es ideal cuando se necesita **control total sobre el flujo** y razonamiento sobre el estado del agente en cada paso.

7 casos de uso de referencia

TRANSVERSALES

- Soporte al cliente
- IT Help Desk
- Analista SOC (seguridad)

VERTICALES

- Servicios financieros
- Triage de salud
- Revisión legal de documentos
- Logística y supply chain

Estos 7 casos son el hilo conductor del curso. Cada módulo los revisita con mayor profundidad técnica. La validación y métricas de estos casos se analizan en el Módulo 9.

Caso: Soporte al cliente con ADK2

```
from google.adk.agents import Agent
from google.adk.tools import FunctionTool

def buscar_ticket(ticket_id: str) -> dict:
    """Busca un ticket de soporte en la base de datos."""
    return crm_api.get_ticket(ticket_id)

def escalar_ticket(ticket_id: str, razon: str) -> str:
    """Escala un ticket a un agente humano."""
    return crm_api.escalate(ticket_id, reason=razon)

soporte_agent = Agent(
    name="soporte_cliente",
    model="gemini-2.5-pro",
    instruction="""Sos un agente de soporte al cliente.
Resolvé consultas usando las herramientas disponibles.
Escalá a humano si la situación lo requiere.""",
    tools=[
        FunctionTool(buscar_ticket),
        FunctionTool(escalar_ticket)
    ]
)
```


Caso: Analista SOC con GCP

El caso del analista de seguridad ilustra **en qué condiciones** un agente aporta algo que un SIEM basado en reglas no puede dar — y en cuáles el SIEM sigue siendo la respuesta correcta:

SISTEMA TRADICIONAL (SIEM)

- Reglas fijas, muchos falsos positivos
- No razona sobre contexto
- Requiere analista para cada alerta
- No correlaciona entre fuentes

AGENTE SOC (CHRONICLE + ADK2)

- Correlaciona múltiples fuentes
- Razona sobre patrones complejos
- Genera hipótesis y las prueba
- Escala solo los casos relevantes

*No es un reemplazo: en producción **conviven**. La detección de alto volumen y baja ambigüedad se queda en reglas — es más barata, más rápida y auditable línea por línea. El agente entra donde la correlación entre fuentes y el contexto hacen inviable enumerar las reglas de antemano.Cuál gana no se decide por dominio sino caso por caso, con el criterio de la slide siguiente.*

Principio: la simplicidad primero

El error más común al adoptar agentes IA es **sobre-ingeniería prematura**:

✗ ANTI-PATRÓN:

"Vamos a hacer un sistema multi-agente con 12 agentes especializados"
(para un problema que un workflow resuelve en 50 líneas de código)

✓ PATRÓN CORRECTO:

1. ¿Código simple resuelve esto? → usar código
2. ¿Workflow con ramas? → usar workflow
3. ¿Solo responder preguntas? → usar RAG
4. ¿Nada de lo anterior funciona? → agente IA

*La **complejidad accidental** es el mayor enemigo de los sistemas agenticos en producción. Cada nivel de agencia agrega latencia, costo, imprevisibilidad y superficie de falla.*

Estándares abiertos: el principio de future-proofing

ESTÁNDARES A ADOPTAR

- **OpenAPI** — definición de herramientas
- **MCP** — protocolo de contexto de modelo
- **A2A** — comunicación entre agentes
- **OAuth 2.0** — autenticación de herramientas

ANTI-PATRONES A EVITAR

- APIs propietarias sin abstracción
- Hardcodear un solo proveedor de LLM
- Formatos de memoria no estándar
- Tooling sin contratos versionados

Agentes en industria regulada: qué te van a pedir compliance y auditoría

Los marcos regulatorios emergentes **no tratan igual** a un sistema determinista que a uno autónomo:

MARCO INTERNACIONAL

- **EU AI Act** — clasifica por riesgo y exige supervisión humana en los usos de alto riesgo
- **NIST AI RMF** — distingue determinista de autónomo con distintos requisitos de control
- A mayor agencia, mayor carga probatoria sobre quien lo despliega

MARCO LOCAL (ARGENTINA)

- **Ley 25.326** de Protección de Datos Personales
- **AAIP** — autoridad de aplicación y guías de IA
- **BCRA** — normativa de gestión de riesgo tecnológico y externalización para entidades financieras
- Verificar la norma vigente con Legales: cambia más rápido que el material de un curso

CONSECUENCIA DE ARQUITECTURA

- La auditabilidad es un **requisito legal**, no una preferencia de ingeniería
- Patrón correcto: lógica de negocio **determinista** en los pasos regulados, LLM solo en los **no** regulados
- La pregunta del auditor no es "¿anda bien?" sino "¿por qué decidió eso, y quién lo firmó?"

IMPLEMENTACIÓN CONCRETA DEL HITL:

En ADK2 el control no es un "nivel de seguridad" global del agente: cada `FunctionTool` acepta `require_confirmation` (booleano o función de los argumentos). Eso permite que `consultar_movimientos` corra sola y `bloquear_tarjeta` exija aprobación humana — **en el mismo agente**. El control queda declarado por herramienta, en código, y por lo tanto es revisable y diffeable.

Unit economics de un agente

Un agente no cuesta "lo que sale el modelo". Cuesta **tokens × llamadas × reintentos**:

```
costo_mes = V × [ T_router × P_barato + T_resolución × P_modelo ] × (1 + tasa_reintento) / 1.000.000
V = consultas/mes      T = tokens por llamada (prompt + tools + respuesta)
P = USD por 1M tokens (COSTOS_MODELO en lab.md, Ej.3)
gemini-2.5-flash  0,075 USD/1M      gemini-2.5-pro  3,50 USD/1M      →  1 pro ≈ 47 flash
```

Ejemplo con supuestos declarados — **V = 50.000/mes**, **T_resolución = 4.000**, **T_router = 300**, **reintentos 15%**, **mix 70/30**:

ARQUITECTURA	LLAMADAS AL MODELO POR CONSULTA	COSTO MENSUAL
Agente, todo con pro	1 (+ reintentos)	≈ 805 USD
Agente con router 70/30	2: clasificar (flash) + resolver	≈ 255 USD
Workflow equivalente (1 llamada flash, ruteo determinista)	1	≈ 5 USD

Los **supuestos** (V, T, reintentos, mix) son de la slide y hay que reemplazarlos por los propios: los precios salen de **lab.md**, el resto no. Lo que mueve la cuenta no es el precio del token sino **cuántas veces se llama al modelo**. Y falta lo que no está en la tabla: latencia, horas de revisión humana, y el costo de correr la evaluación en cada build (Módulo 9). Un agente que sale 50× más que el workflow se justifica solo si resuelve casos que el workflow pierde — y eso hay que *medirlo*, no suponerlo.

Cómo se conecta este módulo con el resto

MÓDULO 2 →

Diseño de componentes core: herramientas, memoria y orquestación en profundidad.

MÓDULO 4 →

MCP (Model Context Protocol): el protocolo de herramientas remotas presentado hoy.

MÓDULO 8 →

A2A (Agent-to-Agent): coordinación y colaboración entre agentes.

Módulo 9 → Validación y métricas de los 7 casos de uso de referencia presentados en este módulo.

5 ideas que se llevan hoy

01

La agencia es un **espectro**.
Calibrar autonomía a
variabilidad de la tarea y
tolerancia al riesgo.

02

Usar la **arquitectura más simple** que funcione: código → workflow → RAG → agente.

03

Diseñar para **multimodelo y estándares abiertos** desde el día uno.

04

Los agentes brillan con **entradas no estructuradas, razonamiento multistep o tareas paralelas**.

05

Los 5 pilares: **escalabilidad, modularidad, aprendizaje continuo, resiliencia, future-proofing**.

PRÓXIMO ENCUENTRO

Módulo 2

Componentes

del Agente

Herramientas · Memoria · Orquestación · Agent Loop

LAB 01 — ENTREGA LA PRÓXIMA CLASE